Computación Concurrente, Paralela y Distribuida, Práctica 6

Balanceo de carga y dependencias

Versión: 12 de febrero de 2026

Contexto y Objetivos

En esta práctica nos vamos a centrar en **dos conceptos clave** del paralelismo que hasta ahora hemos mayormente ignorado debido a que, por diseño, se ha evitado su presencia en los códigos al tener un **impacto negativo en el rendimiento**:

- Por un lado veremos el efecto que tiene una **diferencia** en la **carga computacional** de las tareas cuando estas son ejecutadas en paralelo. En **prácticas anteriores**, y dejando de lado el uso de un pipeline, todas las tareas tenían la **misma complejidad** por lo que el reparto con la técnica de *Pooling* causaba que los tiempos de procesamiento fueran muy similares entre procesos.

  En **esta práctica** no obstante se estudiará un escenario de tareas ejecutadas en paralelo, pero analizando los efectos que se dan cuando hay un *desbalanceo de carga* presente debido a una **diferencia** en su **complejidad computacional**. De igual forma se estudiará cómo una elección a la hora de repartir las tareas (**mapeo del procesamiento**) puede evitar, o potenciar, dicha diferencia de carga computacional presente.

- Por otro lado veremos también el papel que tienen las *dependencias de datos* en la paralelización de un código secuencial en origen. Estas dependencias pueden o bien causar **problemas** que obliguen a la **serialización** de partes del código, con la consecuente pérdida de rendimiento, o ser **fuente** potencial de **errores de validación** al paralelizar. Además, se estudiará también las *dependencias de resultados*, las cuales son más problemáticas al **no permitir** de ninguna forma la **paralelización**, siendo por ello crítico saber detectarlas.

Índice

1 Balanceo de carga **2**

2 Dependencias de datos y resultados **6**

Índice de tareas

Tarea 1: Balanceo de carga, implementación de las 4 funciones de paralelización 2
Tarea 2: Balanceo de carga, experimentos 3
Tarea 3: Dependencias de datos, análisis del código secuencial 6
Tarea 4: Dependencias de datos, análisis del código paralelo 7
Tarea 5: Dependencias de datos, ejecución de los códigos 7
Tarea 6: Dependencias de resultados, análisis de código y ejecución 8

Índice de figuras

1 Diagrama de reparto de tareas con *chk_par_ordered* → *se procesan bloques en paralelo, función a función* . . 3
2 Diagrama de reparto de tareas con *fun_par_ordered* → *se procesan funciones en paralelo, bloque a bloque* . . 3
3 Diagrama de reparto de tareas con *fun_par_unordered* y *chk_par_unordered* → *se procesan tareas del producto cartesiano bloques* × *funciones, sin orden* 4
4 Diagrama de la gestión de dependencias de datos 6
5 Diagrama de la gestión de dependencias de resultados, la cual imposibilita el paralelismo 7

Antes de empezar: Para esta práctica se permite el uso de la MR. Aunque el objetivo sea más bien conceptual, para el caso de estudio del desbalanceo de carga se puede usar para hacer ejecuciones más intensivas a fin de ver diferencias de rendimiento entre distintos escenarios.

1. Balanceo de carga

El análisis de la *complejidad computacional* es un paso previo fundamental antes de proceder a la paralelización de cualquier programa o función en el procesamiento de un conjunto de datos. Este análisis nos puede hacer ver si merece la pena aplicar dicha paralelización o no.

Por un lado, en el caso de programas o funciones que hagan procesamiento muy simple, como por ejemplo unas pocas operaciones matemáticas, será más difícil justificar la paralelización por su potencial baja complejidad (e.g., $O(1)$). Esta baja complejidad puede forzar a hacer una división de los datos en grandes bloques, como hemos hecho en prácticas anteriores, para poder así compensar el gasto de paralelización (e.g., comunicaciones, creación del *Pool*), o incluso resultar en que la paralelización no sea beneficiosa.

Por otro lado, cuando tenemos programas o funciones que realizan procesamiento muy pesado para cada dato individual, como por ejemplo aquellas que llevan a cabo una búsqueda en un espacio de variables (e.g., $O(n)$, $O(n^2)$), será más fácil justificar su paralelización incluso sin agrupar datos. Veremos ejemplos de este último caso en prácticas siguientes.

En esta prácticas seguiremos trabajando con conjuntos de números, es decir de grupos de datos individuales, pero les aplicaremos tres funciones con distintas complejidades:

- *fun_cheap*: Una función con **muy baja complejidad**, repite un mismo cálculo, relativamente sencillo, **N_REPEATS** veces. Su complejidad sería así aproximadamente $O(N_REPEATS)$, es decir $O(1)$.
- *fun_medium*: Una función que comprueba si un número es primo, una operación que *per se* tiene una **complejidad variable**, pero que está acotada en su peor caso por $O(n)$.
- *fun_expensive*: Una función costosa que comprueba si un número y su reverso son primos. La complejidad de esta función es también variable, pero puede ser en el peor de los casos de $O(2n)$.

Si aplicamos nuestros conocimientos adquiridos hasta ahora, sabemos que podemos **particionar** este problema en **dos dimensiones**, tanto en las **funciones** a aplicar como en los **bloques de datos** (*chunks* en inglés). No obstante, también podemos considerar que en el **reparto** de tareas y en su ejecución, puede haber un **orden** o no haberlo. Este orden significa que si se particiona por funciones se aplica una función a todos los bloques antes de pasar a otra función, o que se aplican todas las funciones a un bloque antes de pasar a otro bloque. De igual forma, si no hay orden una función puede empezar a aplicarse antes de que acabe la anterior, y un bloque puede empezar a procesarse antes de acabar con el anterior. Esto nos da cuatro maneras posibles de particionar y repartir las tareas para su ejecución:

- Cojo una función y se la aplico a los bloques en paralelo (en orden) $\rightarrow$ *chk_par_ordered*, ver Figura 1
- Cojo un bloque y le aplico todas las funciones en paralelo (en orden) $\rightarrow$ *fun_par_ordered*, ver Figura 2
- Cojo una función y se la aplico a los bloques en paralelo (sin orden) $\rightarrow$ *chk_par_unordered*, ver Figura 3
- Cojo un bloque y le aplico todas las funciones en paralelo (sin orden) $\rightarrow$ *fun_par_unordered*, ver Figura 3

En este caso la imposición de un orden es meramente artificial a efectos académicos, pero es fácilmente justificable esta necesidad en un escenario donde los datos tuvieran *dependencias*, es decir, donde un bloque de datos dependiera de los resultados del bloque anterior. De igual forma, es interesante mencionar que al eliminar la restricción del orden, las funciones de paralelización ***chk_par_unordered* y *fun_par_unordered* serán** en esencia **iguales**.

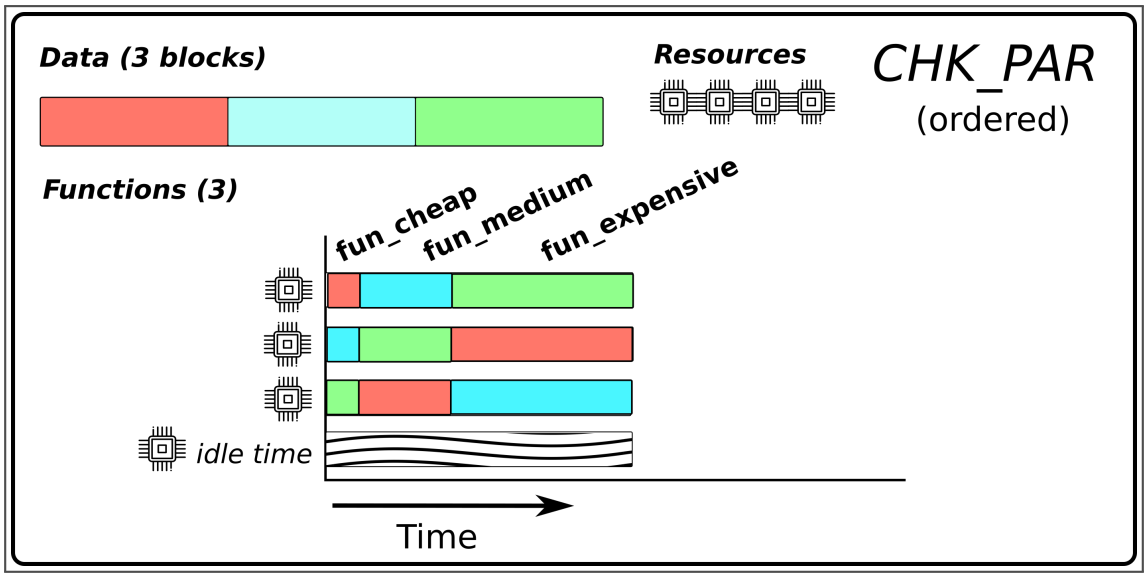

Figura 1: Diagrama de reparto de tareas con *chk_par_ordered* → *se procesan bloques en paralelo, función a función*

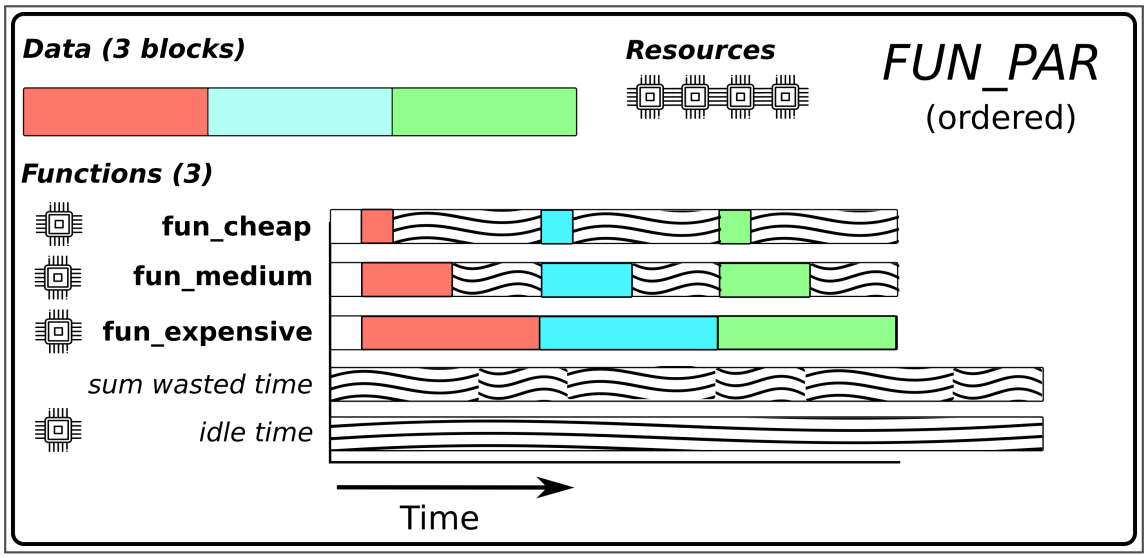

Figura 2: Diagrama de reparto de tareas con *fun_par_ordered* → *se procesan funciones en paralelo, bloque a bloque*

Tarea 1 – Balanceo de carga, implementación de las 4 funciones de paralelización

Observa en el *código1* cómo se han implementado las 4 funciones anteriormente mencionadas. Las diferencias entre ellas son extremadamente sutiles, más si se tiene en cuenta la sintaxis de Python (fíjate en la indentación), no obstante el comportamiento del código es radicalmente distinto.

Complejidad algorítmica y balanceo de carga: Aunque es cierto que la complejidad algorítmica de una función como *fun_medium* con O(n) sea la misma que otra como *fun_expensive* que tiene una de O(2n) en el ámbito académico/teórico, en el caso de una ejecución real esta diferencia sí que causará una desbalanceo de carga de específicamente el doble. Esto hace que aunque el análisis algorítmico sea importante, también lo sea la medición empírica y el análisis individualizado de tiempos para las funciones.

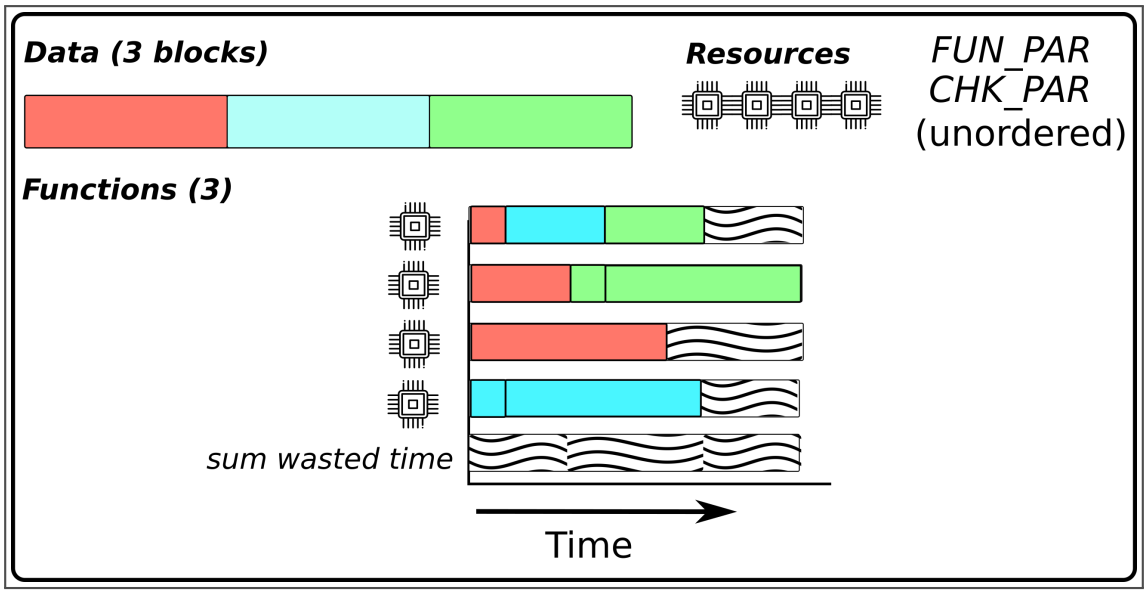

Figura 3: Diagrama de reparto de tareas con *fun_par_unordered* y *chk_par_unordered* → *se procesan tareas del producto cartesiano bloques × funciones, sin orden*

Tarea 2 – Balanceo de carga, experimentos

En el *código1* encontrarás 2 experimentos:

- El primero realiza el procesamiento de las 3 funciones originalmente descritas usando los 4 escenarios de paralelización.
- El segundo realiza el procesamiento pero repitiendo varias veces la misma función, en este caso la *fun_expensive*, a fin de simular un menor desbalanceo de carga pero exponer otras limitaciones.

Ejecuta ahora este código. El resultado para ambos experimentos son 2 tablas, una superior con los tiempos de ejecución (runtimes), y otra inferior con el grado de aprovechamiento de dicho tiempo, es decir, el porcentaje de ese tiempo en el que hubo procesamiento.

Del 1º experimento:

- **¿Cuál es el límite de paralelismo para la función *chk_par_ordered*?**
- **¿Y para la *fun_par_ordered*?**
- **¿Cómo es el rendimiento de la ejecución para la función *fun_par_ordered*?** (pista: compara los tiempos con el de la ejecución secuencial) **¿Por qué se dan estos resultados?**
- **Son los tiempos de las funciones '*fun_par_unordered*' y '*chk_par_unordered*' iguales? ¿Cuál es su límite de paralelismo?**
- Observa la relación entre el aprovechamiento y los tiempos y los límites de paralelismo de cada función de paralelización.

Del 2º experimento:

- **¿Cuál es el límite de paralelización ahora de la función *fun_par_ordered*?**
- Cuando ya no hay desbalanceo de carga, **¿mejoran los grados de aprovechamiento? ¿A qué crees que se puede deber?**

Complejidad algorítmica en base a los datos: Es interesante destacar que la complejidad de las funciones, como las usadas en esta práctica, puede también depender de la entrada de datos, en este caso, de la 'magnitud' o 'escala' de los números. Esto quiere decir que con números 'pequeños' las funciones tendrán unos tiempos de ejecución probablemente rápidos (e.g., comprobar si es primo un número de 4 dígitos lleva x milisegundos), mientras que con números 'grandes' los tiempos serán proporcionalmente más largos (e.g., comprobar un número de 5 dígitos puede llevar $10 \times x$ milisegundos). Sabiendo esto, para modular la entrada de datos en este programa se usan 2 variables:

- **N_BASE**: Número a partir del cual generaremos los **N_LEN** números siguientes, en orden y 1 a 1.
- **N_LEN**: Cuantos números generaremos.

2. Dependencias de datos y resultados

Aunque en las prácticas anteriormente vistas se haya dejado de lado las dependencias de datos, a fin de poder estudiar más a fondo otros conceptos como los límites del paralelismo o la escalabilidad, no son raros los escenarios de tratamiento de datos en los que hay presentes *dependencias de datos*, es decir, en los que el procesamiento de unos datos depende de resultados del procesamiento de datos anteriores (*dependencia de resultados*), o requiere de datos previos o 'adyacentes' (*dependencia de datos* propiamente).

Podemos ver este concepto usando las mismas listas de números que hemos estado usando hasta ahora como datos con dependencias potenciales. En los códigos 2 y 3 encontrarás programas muy simples que procesan unas listas de números y las devuelven actualizadas. No obstante, para realizar las operaciones que en prácticas anteriores solo usaban el propio número, ahora vamos a necesitar el número inmediatamente anterior en el caso del código2 (1 dependencia), y el anterior y posterior en el caso del código3 (2 dependencias). En los casos especiales de números en la 'frontera', es decir el primero y último de la lista, haremos uso de las llamadas semillas ('seeds'), es decir números previamente especificados. Puedes ver todos estos conceptos y un ejemplo con 2 dependencias en la Figura 4.

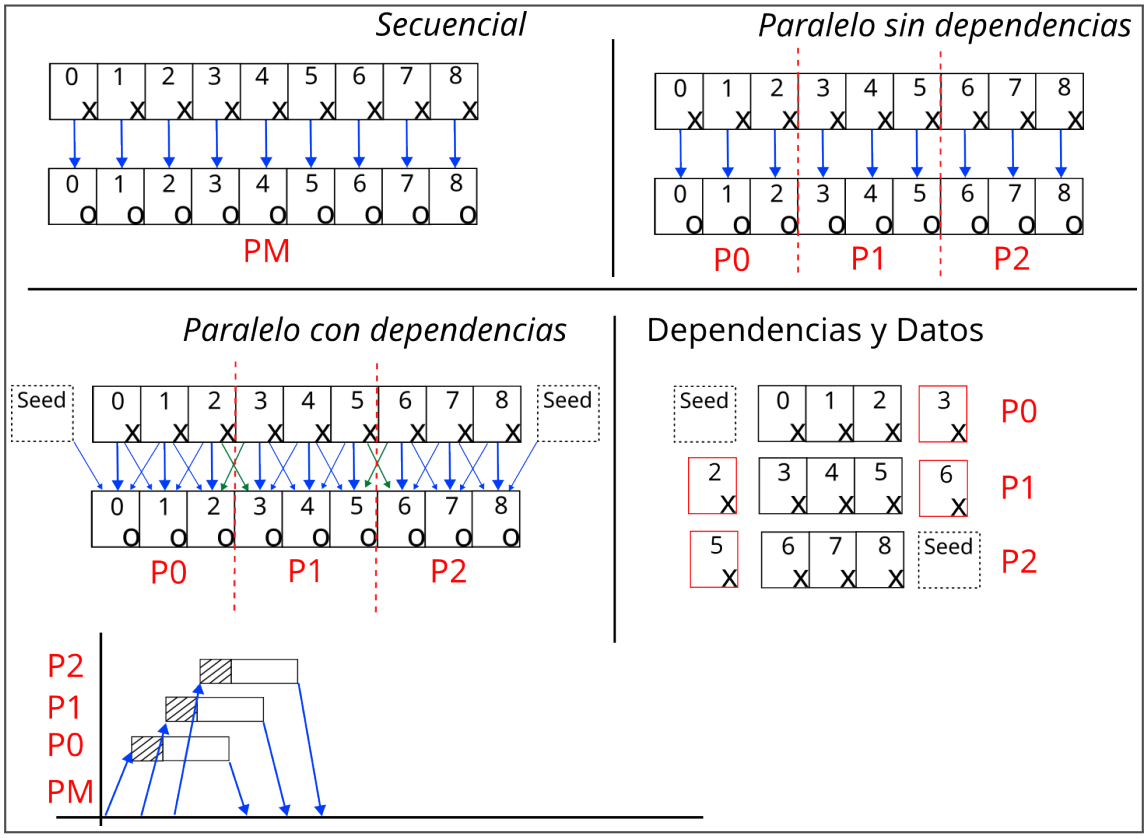

Figura 4: Diagrama de la gestión de dependencias de datos

Es importante destacar que por implementación del problema en estos códigos las dependencias son de datos, NO de resultados. Para acabar la práctica y ver un ejemplo de este último tipo de dependencias, donde la paralelización de este programa sería mucho más compleja, por no decir imposible sin recurrir a técnicas de especulación de resultados, puedes analizar el código 4. También pueds ver una representación gráfica de este problema en la Figura 5.

En general, el manejo de dependencias es potencialmente trivial en la implementación secuencial, dado que el programa tiene acceso a todo el conjunto de datos, pero pasa a jugar un papel esencial en la paralelización. Además, siendo una tarea que no aportará ninguna ganancia *per se* a la paralelización, sí que es muy a menudo una fuente de pérdidas de rendimiento y de errores de programación.

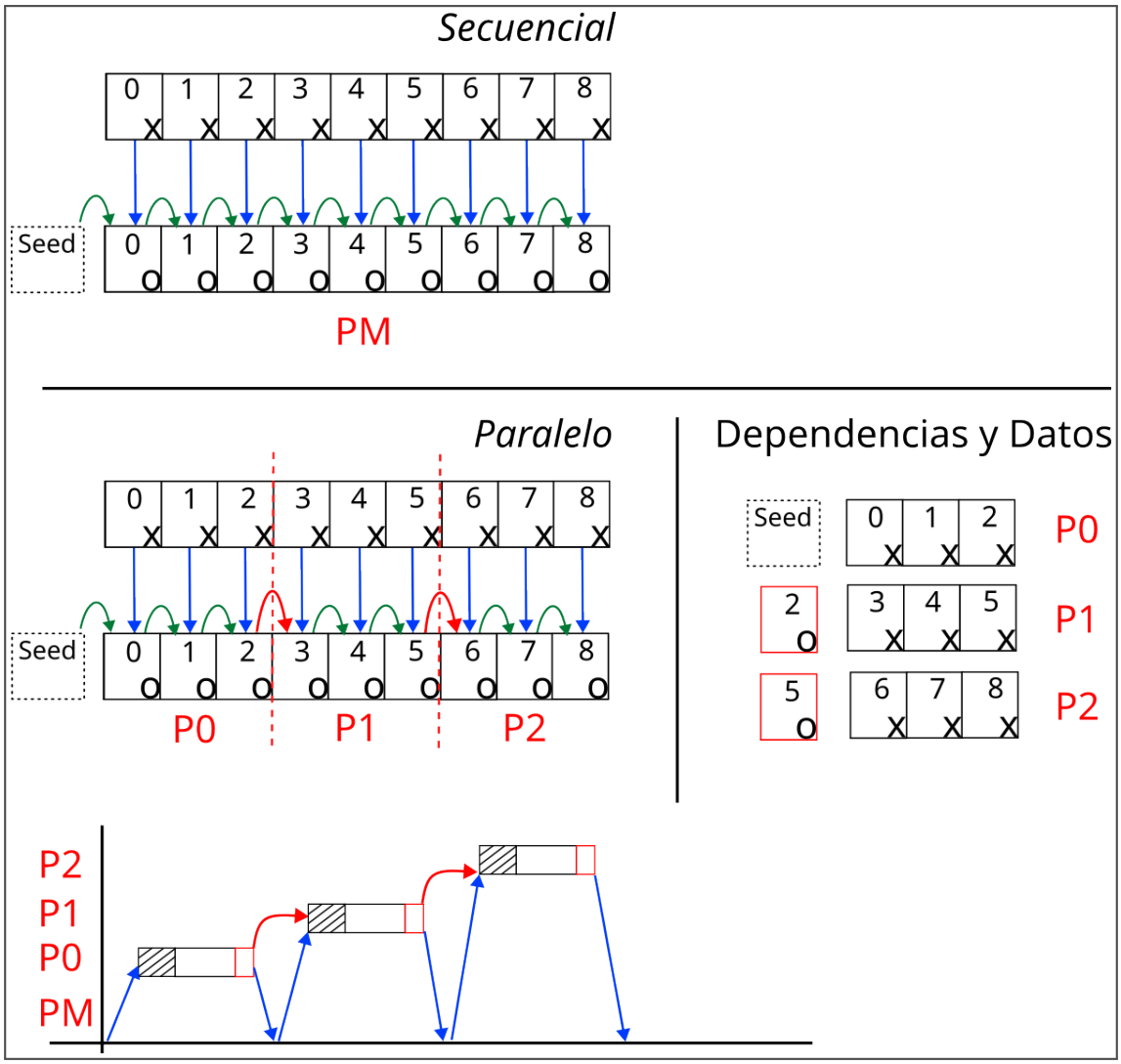

Figura 5: Diagrama de la gestión de dependencias de resultados, la cual imposibilita el paralelismo

Tarea 3 – Dependencias de datos, análisis del código secuencial

Observa en los códigos 2 y 3, el funcionamiento de un programa que procesa una lista de números, pero con dependencias. **Fíjate en la función de procesamiento secuencial.**

Tarea 4 – Dependencias de datos, análisis del código paralelo

Observa ahora en el código2 la implementación en paralelo de este procesamiento. Fíjate en la gestión de la 'semilla', que aunque inicialmente toma el valor de la semilla original (usada por el 1º elemento de la lista, es decir el 1º elemento del 1º bloque), luego toma otros valores a fin de representar la 'dependencia', es decir el dato a 'comunicar' entre bloques. En este último escenario la semilla deberá de ser el último elemento (no resultado) de cada bloque, a fin de ser usado para el procesamiento del siguiente bloque.

Tarea 5 – Dependencias de datos, ejecución de los códigos

Ejecuta los códigos 2 y 3 y observa la salida.

- ¿Han sido los datos troceados y repartidos adecuadamente?
- ¿Es el resultado final válido si se compara con la ejecución secuencial?
- Observando la salida del experimento realizado sin mostrar salida y con un tamaño de datos mucho más grande. A pesar de las dependencias y de su gestión, ¿Hay una ganancia de rendimiento?

Tarea 6 – Dependencias de resultados, análisis de código y ejecución

Trabajando ahora con el codigo4, en el cual se procesa una conjunto de números teniendo en cuenta dependencias de datos y también de resultados:

- Analiza en particular la implementación de la función *sequential* y compárala con su implementación en anteriores códigos.
- ¿Crees que sería posible de manera técnica una paralelización?
- Observa su salida

Copyright & Disclaimer


```
# Copyright © 2025 by Jonatan Enes (jonatan.enes@udc.es)
# Computer Engineering department, Universidade da Coruña, Spain.
#
# This file is part of several courses on parallel processing from Universidade da Coruña,
# and it can only be used and/or modified by the author, the students or any
# explicitly authorized person by the author, inside the context of the subject.
# Redistribution to a third-party without previous authorization is strictly forbidden.
# No commercial use is allowed.
#
# This file has only academic purposes and should not be used for any real-world
# scenario, the author holds no accountability for its use or misuse.
```